ESP32 setup in Arduino IDE – MacBook

Step 1: Add ESP32 board URL

1. Open Arduino IDE.

2. Go to Arduino IDE → Settings.

3. Find Additional Boards Manager URLs.

4. Paste this official URL:

`https://espressif.github.io/arduino-esp32/package_esp32_index.json`

5. Click OK.

Step 2: Install ESP32 board

1. Open Boards Manager from the left sidebar.

2. Search `esp32`.

3. Select esp32 by Espressif Systems.

4. Click Install.

5. Wait until installation completes.

If installation succeeds, continue to Step 3.

If an error appears:

- Check your internet connection.
- Close and reopen Arduino IDE.
- Try installing again.
- If it still fails, take a screenshot and send it to me. Don't randomly install other packages.

Step 3: Connect physical ESP32

1. Connect ESP32 to your Mac using a USB data cable.
2. Open Arduino IDE.
3. Click Tools → Board.
4. Select ESP32 Arduino → ESP32 Dev Module.
5. Open Tools → Port.
6. Select the port corresponding to your ESP32.

If no port appears:

- Try another USB data cable.
- Try another USB port or adapter.

- Check whether your board requires a USB-to-serial driver (CP210x or CH340).

- Restart Arduino IDE after installing the appropriate driver.

Step 4: Enter your Practical 1 code

Paste this code into the editor:

```
const int potPin = 34;
int potValue = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);
}

void loop() {
  potValue = analogRead(potPin);

  if (potValue > 2000) {
    Serial.println("WARNING!");
  }

  Serial.print("Potentiometer Value: ");
  Serial.println(potValue);

  delay(500);
}
```

Step 5: Verify the code

Click the Verify button.

This checks whether your code has any compilation errors.

- If it shows `Done compiling`, your code is correct.

- If it shows `Compilation error`, check the error message.

Step 6: Upload the code

1. Click the Upload arrow.

2. Wait for compilation and uploading.

3. If uploading succeeds, you should see a completion message.

If you see `Connecting...` continuously:

- Hold the BOOT button on your ESP32.

- Click Upload.

- Release BOOT when uploading begins.

The exact procedure can vary by ESP32 board.

Step 7: Check your output

1. Open Tools → Serial Monitor.

2. Set baud rate to `115200`.

3. Rotate your potentiometer.

4. Observe the changing values.

Expected output:

```
Potentiometer Value: 1200
```

```
Potentiometer Value: 1800
WARNING!
Potentiometer Value: 2500
```

```
https://arduino.esp8266.com/stable/
package_esp8266com_index.json

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-
pages/package_esp32_index.json

https://dl.espressif.com/dl/package_esp32_index.jsor
```

```
VCC (outer pin)
```
3V3

SIG (middle pin)

GPIO 34

GND (other outer pin)

GND

Practical 2

PIR sensor	Arduino
VCC (+)	5V
OUT	Pin 2
GND	GND

2. Correct the LED connection

Make sure the LED circuit follows this exact order:




```
int pirPin=2,ledPin=13;
void setup(){
  pinMode(pirPin,INPUT);
  pinMode(ledPin,OUTPUT);
  Serial.begin(9600);
}
void loop(){
  int motionState=digitalRead(pirPin);
  if(motionState==HIGH){
    digitalWrite(ledPin,HIGH);
    Serial.println("Motion detected!");
  }
  else{
    digitalWrite(ledPin,LOW);
```

```
    Serial.println("No motion detected");
  }
  delay(1000);
}

  else{
    digitalWrite(ledPin,LOW);
    Serial.println("No motion detected");
  }
  delay(1000);
}
```

PRACTICAL 3

4. Complete connection table

Component	Connect to
GPIO 12	First resistor
First resistor	Red LED long leg (+)
Red LED short leg (–)	GND
GPIO 14	Second resistor
Second resistor	Green LED long leg (+)
Green LED short leg (–)	GND

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);

int redLED = 12;
int greenLED = 14;

void setup() {
  Serial.begin(115200);

  pinMode(redLED, OUTPUT);
  pinMode(greenLED, OUTPUT);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected!");
  Serial.println(WiFi.localIP());

  server.on("/", []() {
```

```
  server.send(200, "text/html",
    "<h1>ESP32 LED Control</h1>"
    "<a href='/red/on'>Red LED ON</a><br>"
    "<a href='/red/off'>Red LED OFF</a><br>"
    "<a href='/green/on'>Green LED ON</a><br>"
    "<a href='/green/off'>Green LED OFF</a>");
});

server.on("/red/on", []() {
  digitalWrite(redLED, HIGH);
  server.send(200, "text/plain", "Red LED ON");
});

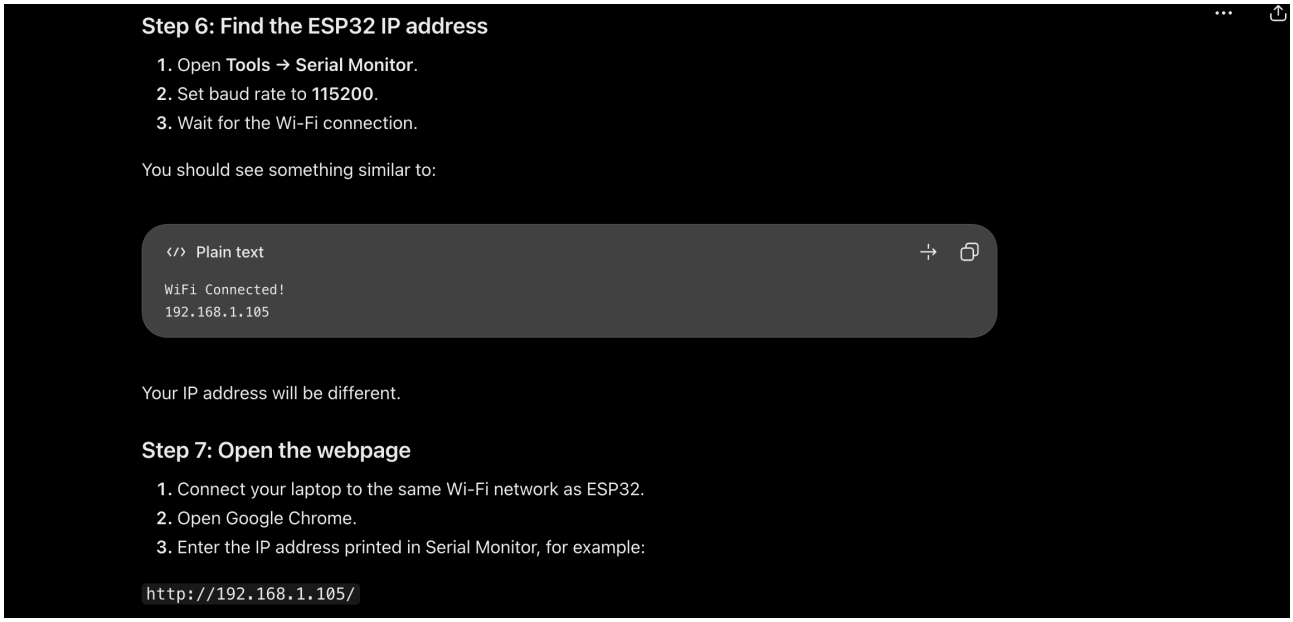
server.on("/red/off", []() {
  digitalWrite(redLED, LOW);
  server.send(200, "text/plain", "Red LED OFF");
});

server.on("/green/on", []() {
  digitalWrite(greenLED, HIGH);
  server.send(200, "text/plain", "Green LED ON");
});

server.on("/green/off", []() {
  digitalWrite(greenLED, LOW);
  server.send(200, "text/plain", "Green LED OFF");
});
```

```
  server.begin();
}

void loop() {
  server.handleClient();
}
```



PRACTICAL 4




```
#include <WiFi.h>
```

```
#include <WebServer.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

const int potPin = 34;
int potValue = 0;

WebServer server(80);

void handleRoot() {
  potValue = analogRead(potPin);

  String webpage = "<html><head>";
  webpage += "<meta http-equiv='refresh' content='1'>";
  webpage += "<title>ESP32 Sensor Data</title>";
  webpage += "</head><body>";
  webpage += "<h1>ESP32 Potentiometer Monitor</h1>";
  webpage += "<h2>Potentiometer Value: ";
  webpage += String(potValue);
  webpage += "</h2>";
  webpage += "</body></html>";

  server.send(200, "text/html", webpage);
}

void setup() {
  Serial.begin(115200);
```

```
  pinMode(potPin, INPUT);

  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, handleRoot);

  server.begin();
  Serial.println("Web Server Started");
}

void loop() {
  potValue = analogRead(potPin);
  server.handleClient();
  delay(100);
}
```